Text Inpainting using Discrete Diffusion (D3PM)

Overview

What is Text Inpainting?

Text inpainting refers to the task of reconstructing missing or masked portions of a sentence or document using surrounding context.

Traditional masked language models such as BERT predict masked tokens in a single step. However, reconstructing long contiguous spans can be difficult because the model must infer multiple tokens simultaneously.

Diffusion-based approaches solve this problem by performing reconstruction gradually. Instead of predicting the full span in one step, the model iteratively denoises a corrupted sequence across multiple diffusion timesteps. This allows the model to progressively refine its predictions and better capture long-range contextual dependencies.

This project implements a **Discrete Diffusion Probabilistic Model (D3PM-style)** for span-based text inpainting using a pretrained BERT backbone.

The development progressed through:

1. Transformer from scratch
2. Pretrained BERT baseline
3. Diffusion without mask conditioning
4. Diffusion with explicit mask conditioning
5. Conditioning dropout
6. Decoding experiments (temperature & top-k)
7. Evaluation (Accuracy, BLEU, ROUGE-L)
8. Interactive Gradio UI

Dataset

We use **WikiText-2 (raw version)** from HuggingFace: WikiText-2 is a widely used benchmark dataset for language modeling tasks. It contains high-quality English text extracted from Wikipedia articles, preserving punctuation and sentence structure.

WikiText-2 is relatively small (~2M tokens). This limitation becomes important when training large models from scratch, which is why pretrained language models such as BERT are later introduced in this project.

```
None
load_dataset("wikitext", "wikitext-2-raw-v1")
```

Original Dataset Sizes

Split	Samples
Train	36,718
Validation	3,760
Test	4,358

After Cleaning

(Removed empty lines, headers, short fragments <10 characters)

Split	Samples
Train	23,547
Validation	2,454
Test	2,850

Preprocessing Pipeline

Preprocessing Motivation

Transformer-based models require tokenized input sequences of fixed length. Raw text data therefore needs to be transformed into structured sequences before training.

The preprocessing pipeline performs several key steps:

1. **Tokenization**
   The pretrained `Bert-base-uncased` tokenizer converts raw text into WordPiece tokens.
2. **Fixed-Length Chunking**
   Text sequences are split into chunks of length 256 tokens to maintain consistent model input size.
3. **Sliding Window Expansion**
   To increase dataset size, overlapping windows are used. This generates additional training sequences while preserving contextual continuity.
4. **Masking Strategy**
   Masking simulates missing information that the model must reconstruct. Two strategies are explored:
   - Random token masking
   - Contiguous span masking

Span masking is significantly more challenging because the model must infer multiple adjacent tokens simultaneously.

Phase 1 — Transformer From Scratch Baseline Model

Motivation for Training a Transformer From Scratch

Before using pretrained models, the project first explores training a transformer encoder directly on WikiText-2. This helps establish a baseline and provides insight into the limitations of training language models with limited data.

The custom transformer includes:

- Multi-head self-attention
- Position-wise feedforward layers
- Residual connections and layer normalization
- Hidden size: 256 → 384
- Layers: 4 → 6
- Attention heads: 4 → 6
- Masked cross-entropy loss

Result

Masked-token accuracy: **~5–6%**

Conclusion

WikiText-2 (~2M tokens) is insufficient to train a strong language model from scratch. This experiment highlights the importance of pretrained models in modern NLP workflows.

Phase 2 — Pretrained BERT Baseline

Why Use Pretrained BERT?

BERT (Bidirectional Encoder Representations from Transformers) is a pretrained language model trained on massive corpora including Wikipedia and BookCorpus.

Key advantages of using pretrained models include:

- Rich contextual embeddings learned from large datasets

- Strong generalization across many NLP tasks
- Faster convergence during fine-tuning

Used:

```
None
BertForMaskedLM.from_pretrained("bert-base-uncased")
```

Fine-tuned on masked token prediction.

Results (9k sequences, 3 epochs, batch=32)

Mask Type	Ratio	Train Accuracy	Validation Accuracy	Train Loss	Validation Loss
Random	0.10	57.83	57.36	2.1694	2.2301
Random	0.25	53.07	51.85	2.4602	2.6114
Random	0.40	45.01	43.90	3.0190	3.2059
Span	0.10	20.45	21.23	5.1082	5.1766
Span	0.25	19.44	19.94	5.0969	5.2521
Span	0.40	17.51	17.91	5.2213	5.4480

Observation:

Random masking is significantly easier than span masking.

Results (73k sequences, 3 epochs, batch=32, span masking, span ratio - 0.25)

Train Loss: **4.5502**
Train Accuracy: **21.79%**
Validation Loss: **5.4015**
Validation Accuracy: **19.89%**

Phase 3 — Diffusion Model (Without Mask Conditioning)

Diffusion Models for Text

Diffusion models were originally introduced for image generation, where noise is gradually added to data and then removed through a learned reverse process.

In the context of text, we cannot add continuous Gaussian noise because tokens are discrete symbols. Instead, discrete diffusion replaces tokens with special mask tokens during the forward corruption process.

The diffusion process consists of two stages:

1. **Forward Process (Corruption)**
   Tokens are progressively replaced with `[MASK]` tokens across multiple timesteps.
2. **Reverse Process (Denoising)**
   The model learns to recover the original tokens step-by-step.

This gradual reconstruction allows the model to generate more coherent text compared to single-step masked token prediction.

Setup 1

- Training:
  - T= 12
  - 6 epochs
  - Batch size: 16
  - Mask ratio: 0.25
  - Mask type: Span
  - 9k sequences

Result

Train Loss: **5.0208**
Train Accuracy: **22.80%**
Validation Loss: **4.9633**
Validation Accuracy: **24.39%**

Setup 2

- Training:
  - T= 12
  - 6 epochs
  - Batch size: 16
  - Mask ratio: 0.25
  - Mask type: Span
  - 73k sequences
  - Gradient accumulation = (effective batch size = 32)

Result

Train Loss: **4.5649**
Train Accuracy: **26.08%**
Validation Loss: **4.9749**
Validation Accuracy: **24.58%**

Phase 4 — Diffusion With Mask Conditioning

Mask Conditioning

During diffusion training, the model receives partially corrupted sequences. However, the model may not always know which tokens were originally masked versus which tokens were introduced during diffusion corruption.

To address this, explicit **mask conditioning** is introduced. A binary mask embedding is added to the input representation to indicate the original masked positions.

This provides the model with additional structural information, helping it focus reconstruction efforts on the relevant tokens.

Mask conditioning significantly improves reconstruction accuracy compared to diffusion without mask awareness.

T = 8 (Span 0.25)

Train Loss: **3.3225**
Train Accuracy: **42.65%**
Validation Loss: **3.4671**
Validation Accuracy: **42.89%**

T = 12 (Best Configuration)

Mask Type	Ratio	Train Accuracy%	Validation Accuracy%	Train Loss	Validation Loss
Random	0.25	71.24	69.50	1.3612	1.5401
Span	0.10	49.84	50.82	2.9007	2.9259

Span	0.25	49.57	48.97	2.8369	3.0832
Span	0.40	46.55	46.49	3.0933	3.2812

Conclusion

Using the span masking with **span ratio of 10% at T = 12** as it performs best in span masking to do furthur experiments.

Conditioning Dropout

Why Conditioning Dropout?

Conditioning dropout randomly disables mask conditioning during training with a certain probability. This encourages the model to rely more on contextual information from surrounding tokens rather than exclusively on the mask indicator.

This technique improves robustness and prevents overfitting to the masking signal. Added conditioning dropout (0.1) to reduce over-reliance on mask embeddings.

Result:

Train Loss: **2.9033**
Train Accuracy: **49.80%**
Validation Loss: **2.9092**
Validation Accuracy: **50.97%**

Final Selected Model

Configuration:

- Mask type: Span
- Mask ratio: 0.10
- Timestep: 12
- Conditioning dropout: 0.1
- Temperature: 0.8
- Top-k: 20

Test Results

Test Loss: **2.9221**
Test Accuracy: **51.14%**

Evaluation Metrics

Multiple evaluation metrics are used to assess model performance:

Accuracy
Measures the percentage of masked tokens correctly predicted.

BLEU Score
BLEU evaluates n-gram overlap between generated text and reference text. While widely used in translation tasks, BLEU can underestimate performance in generation tasks where multiple valid outputs exist.

ROUGE-L
ROUGE-L measures the longest common subsequence between generated and reference text. It better captures structural similarity and sequence-level coherence.

For text inpainting, ROUGE-L often provides a more meaningful evaluation than BLEU.

Metric	Score
Masked BLEU	0.0826
Masked ROUGE-L	0.3484
Validation Accuracy	50.97%

BLEU remains low due to strict n-gram matching.
ROUGE-L better captures structural similarity in span reconstruction.

Inference Decoding Experiments

Decoding Strategy

During inference, the model generates tokens using probabilistic sampling. Two key parameters control generation behavior:

Temperature
Temperature scales the output probability distribution.

- Low temperature → deterministic predictions
- High temperature → more diverse but potentially noisy predictions

Top-k Sampling
Top-k sampling restricts token selection to the k most probable candidates. This prevents extremely unlikely tokens from being sampled while still allowing some diversity.

The combination of temperature and top-k helps balance fluency and creativity during text generation.

Tested:

- Temperature: 0.8, 1.0, 1.2
- Top-k: 0, 20, 50

Best decoding configuration:

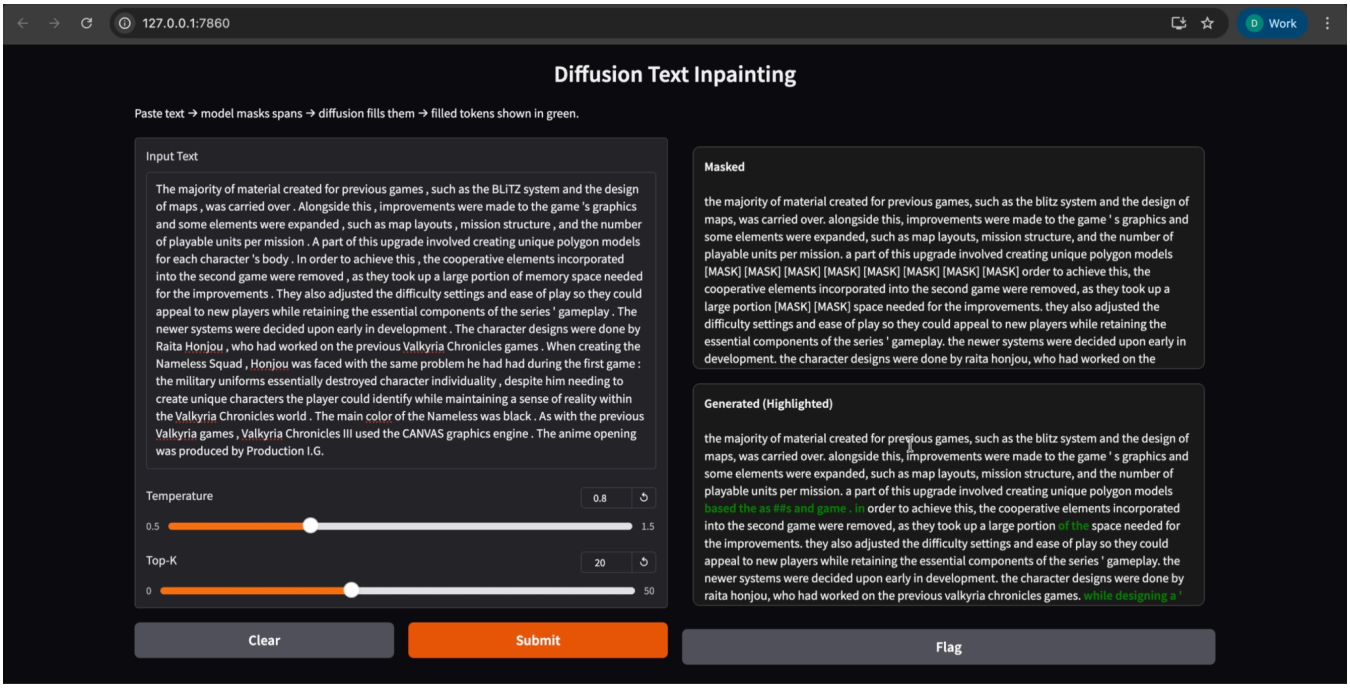
```
None
Temperature = 0.8
Top-k = 20
```

Balanced diversity and coherence.

Gradio UI

Interactive interface allows:

- Paste input text
- Auto span masking
- Diffusion-based reconstruction
- Highlight reconstructed tokens
- Adjustable temperature
- Adjustable top-k



[Gradio UI Video]

Folder Structure

```
None
TEXT-INPAINTING_02/
│
├── data/
│   ├── dataset.py
│   ├── diffusion_dataset.py
│   ├── load_data.py
│   ├── masking.py
│   └── preprocessing.py
│
├── diffusion/
│   └── forward_process.py
│
├── evaluation/
```

```
│   ├── bleu.py
│   ├── metrics.py
│   └── rouge.py
│
├── inference/
│   ├── inpaint.py
│   └── reverse_diffusion.py
│
├── models/
│   ├── diffusion_model.py
│   └── transformer.py
│
├── notebooks/
│   └── 01_data_analysis.ipynb
│
├── training/
│   ├── diffusion_trainer.py
│   ├── loss.py
│   └── trainer.py
│
├── utils/
│   ├── device.py
│   └── seed.py
│
├── app.py
├── main.py
├── requirements.txt
├── README.md
└── .gitignore
```

Folder Responsibilities

`data/`
Handles dataset loading, preprocessing, tokenization, and masking logic.

`diffusion/`
Implements the forward corruption process for discrete diffusion.

`evaluation/`
Implements:

- BLEU score
- ROUGE-L
- Accuracy metrics

`inference/`
Reverse diffusion sampling and inpainting pipeline.

`models/`
Contains:

- Baseline Transformer
- DiffusionBert model with timestep + mask conditioning

`notebooks/`
Analysing the dataset

`training/`
Training loops for:

- Baseline model trainer
- Diffusion model trainer
- Cross entropy loss function

`utils/`
Utility functions:

- Device management (CPU / MPS)
- Seed setting for reproducibility

`app.py`
Gradio-based UI for interactive text inpainting.

`main.py`
Entry point for:

- Training
- Validation
- Testing
- BLEU / ROUGE evaluation
- Inference mode

Requirements

```
None
torch
transformers
datasets
tokenizers
numpy
pandas
tqdm
gradio
nltk
rouge-score
```

Final Outcome

Target: **35%+ masked-token accuracy**

Final Achieved: **50%+ masked-token accuracy**

This project demonstrates a complete diffusion-based text inpainting pipeline with training, evaluation, and interactive deployment.